TRAILBLAZE

Claude Code

Tips & Best Practices

2026-03-30

The 10 Tips That Matter Most

1. **Use the best model with thinking**

   A wrong fast answer is slower than a right slow answer. Use Opus + extended thinking. The opusplan alias auto-switches: Opus during Plan Mode, Sonnet for execution.

   See: Boris Cherny's Workflow

2. **Give Claude a way to verify its work**

   Tests, screenshots, linters -- any feedback loop. This alone 2-3x quality.

   See: Verification

3. **Be specific in your prompts**

   Reference files with @, paste screenshots, point to patterns. Vague = wasted tokens.

   See: Prompting & Context

4. **Plan first, code second**

   Start in Plan Mode (Shift+Tab x2). Iterate until solid, then auto-accept.

   See: Plan Mode

5. **Invest in your CLAUDE.md**

   Every mistake becomes a rule. Check it into git. The whole team contributes.

   See: The CLAUDE.md System

6. **Use /clear between tasks**

   Context degrades as the window fills. Clear aggressively between tasks.

   See: Context Management

7. **Parallelize everything**

   Run 3-5+ sessions with claude --worktree. The single biggest productivity unlock.

   See: Parallel Sessions & Subagents

8. **Use subagents for research**

   Delegate investigation to subagents. Your main context stays clean.

   See: Parallel Sessions & Subagents

9. **Build reusable slash commands**

   Convert repeated workflows into skills in .claude/skills/. Commit to git.

   See: Skills, Hooks & MCP

10. **Use TDD for bug fixes**

    Write a failing test first, then let Claude fix it. Tests constrain scope and prevent regressions.

    See: Test-Driven Development

Plan Mode: Think Before You Code

> *"Most of my sessions start in Plan mode. If the goal is a Pull Request, I use Plan mode and go back and forth with Claude until I like its plan. From there, I switch into auto-accept mode and Claude can usually one-shot it."*
>
> -- Boris Cherny, Claude Code creator

Plan Mode restricts Claude to read-only operations -- it explores your codebase and creates implementation plans without writing code. Toggle with Shift+Tab (cycles Normal -> Auto-Accept -> Plan) or type /plan.

The Four-Phase Workflow

1. Explore (Plan Mode)

Have Claude read and understand the relevant code.

```
read /src/auth and understand how sessions work.
also check how we manage environment variables.
```

2. Plan (Plan Mode)

Ask for a detailed plan. Press Ctrl+G to edit it in your text editor.

```
I want to add Google OAuth. What files need to change?
What's the session flow? Create a plan.
```

3. Implement (Normal Mode)

Switch back (Shift+Tab). Let Claude code, verifying against its plan.

```
Implement the OAuth flow from your plan.
Write tests, run them, and fix any failures.
```

4. Commit

```
Commit with a descriptive message and open a PR.
```

TIP

Skip Plan Mode for small, clear tasks (typos, log lines, renames). Use it when the approach is uncertain, changes span multiple files, or you're unfamiliar with the code.

Advanced: Two-Session Planning

Have one Claude write the plan, then deploy a second instance to review it as a staff engineer. When issues arise, return to Plan Mode rather than pushing forward.

Verification: Give Claude a Feedback Loop

> *"Give Claude a way to verify its work. If Claude has that feedback loop, it will 2-3x the quality. This is probably the most important thing."*
>
> -- Boris Cherny, Claude Code creator

Claude performs dramatically better when it can check its own work. Without clear success criteria, you become the only feedback loop.

Test-Driven Verification

Include test cases so Claude verifies after implementing:

```
Write a validateEmail function.
Test cases: user@example.com -> true,
invalid -> false, user@.com -> false.
Run the tests after implementing.
```

Visual Verification

Paste a screenshot and ask Claude to compare:

```
[paste screenshot] Implement this design.
Take a screenshot of the result and compare
it to the original. List differences and fix them.
```

Root Cause Verification

```
The build fails with this error: [paste error].
Fix it and verify the build succeeds.
Address the root cause, don't suppress the error.
```

Challenge Claude as Reviewer

Flip the dynamic -- ask Claude to review rigorously:

- "Grill me on these changes"
- "Prove to me this works"
- "Review for edge cases, race conditions, and security issues"
- "What's the behavioral diff between this branch and main?"

> **TIP**
>
> Verification can be a test suite, linter, bash command, or browser extension. The Claude in Chrome extension opens tabs, tests UI, and iterates until it works.

The CLAUDE.md System

> *"Invest in your CLAUDE.md. Anytime you see Claude do something incorrectly, add it so Claude knows not to do it next time. The file compounds in value over time."*
>
> -- Boris Cherny, Claude Code creator

CLAUDE.md is read at the start of every conversation. It stores persistent context Claude cannot infer from code alone.

File Hierarchy

- ~/.claude/CLAUDE.md -- Global: all sessions
- ./CLAUDE.md -- Project root: check into git, shared with team
- ./CLAUDE.local.md -- Local only (.gitignore this)
- .claude/rules/*.md -- Topic-specific rule files; path-scoped rules only load when Claude works with matching files

- Parent directories -- Useful for monorepos
- Auto memory (~/.claude/projects/) -- Claude writes its own notes; manage with /memory

TIP

Run /init to auto-generate a starter CLAUDE.md. It analyzes your codebase to detect build systems, test frameworks, and code patterns. Use it as a baseline, then refine over time.

What to Include

- Bash commands Claude can't guess (build, test, lint)
- Code style rules that differ from defaults
- Testing instructions and preferred runners
- Repo etiquette (branch naming, PR conventions)
- Architectural decisions and common gotchas

What NOT to Include

- Things Claude can figure out by reading code
- Standard conventions Claude already knows
- Detailed API docs (link instead), long tutorials

Example

```
# Code style
- Use ES modules (import/export), not CommonJS
- Destructure imports when possible

# Workflow
- Typecheck when done making code changes
- Prefer running single tests, not the whole suite

# Build
- Test: npm run test -- --testPathPattern=<file>
- Lint: npm run lint
```

Importing Other Files

```
See @README.md for project overview.
Git workflow: @docs/git-instructions.md
```

TIP

Keep it concise. For each line, ask: 'Would removing this cause mistakes?' If not, cut it. Bloated files cause Claude to ignore your instructions. Use IMPORTANT or YOU MUST for critical rules.

Context Management

Most best practices reduce to one constraint: Claude's context window fills up fast, and performance degrades as it fills.

The Core Commands

```
/clear              Reset context entirely
/compact [focus]     Compress, optionally keep a focus
/context             Visualize current context usage
/btw [question]      Ask a side question without adding to context
/statusline          Set up the status line UI
```

When to Clear

- Between unrelated tasks -- always
- After two failed corrections -- start fresh with a better prompt
- When Claude starts 'forgetting' instructions
- Before starting a new feature or bug fix

Partial Compaction

Esc+Esc (or /rewind), select a checkpoint, choose 'Summarize from here'. Or: /compact Focus on the API changes -- directs what to keep.

Course-Correct Early

- Ctrl+C -- Stop Claude mid-action (context preserved)
- Esc+Esc or /rewind -- Restore previous state
- "Undo that" -- Revert changes

Sessions & Checkpoints

```
claude --continue    # Resume most recent conversation
claude --resume      # Pick from recent sessions
/rename my-feature   # Name sessions
```

Every action creates a checkpoint. Checkpoints persist across sessions -- close your terminal and rewind later. Try risky things freely.

Prompting & Context

The more precise your instructions, the fewer corrections you need.

Good vs. Bad Prompts

Vague: "add tests for foo.py"
Better: "Write a test for foo.py covering the edge case where the user is logged out. Avoid mocks."

Vague: "fix the login bug"
Better: "Users report login fails after session timeout. Check src/auth/, especially token refresh. Write a failing test that reproduces the issue, then fix it."

Vague: "add a calendar widget"
Better: "Look at how existing widgets are implemented. HotDogWidget.php is a good example. Follow the pattern."

Ways to Provide Rich Context

- Use @ to reference files -- Claude reads them before responding
- Paste images directly (Ctrl+V)
- Give URLs for documentation and API references
- Pipe data in: cat error.log | claude
- Tell Claude to fetch context itself using bash or MCP tools

Let Claude Interview You

For larger features, have Claude interview you to surface edge cases and tradeoffs.

```
I want to build [brief description].
Interview me using the AskUserQuestion tool.
Dig into edge cases and tradeoffs.
Then write a complete spec to SPEC.md.
```

TIP

Once the spec is done, start a fresh session to execute it. Clean context focused entirely on implementation.

Parallel Sessions & Subagents

> *"I run 5 Claude instances in parallel in my terminal, and 5-10 Claudes on claude.ai/code. I treat AI as a capacity you schedule."*
>
> -- Boris Cherny, Claude Code creator

Git Worktrees (the #1 Productivity Unlock)

Claude Code has native worktree support via the --worktree (-w) flag. Each worktree gets its own working directory and branch -- no file collisions between parallel sessions. Repository history and remotes are shared.

```
# Start Claude in a named worktree
# Creates .claude/worktrees/feature-auth/ with branch worktree-feature-auth
claude --worktree feature-auth

# Auto-generate a random name (e.g. bright-running-fox)
claude --worktree

# Run multiple worktree sessions in parallel
claude -w feature-auth &
claude -w bugfix-123 &
```

- Worktrees live at <repo>/.claude/worktrees/<name>
- Branch is named worktree-<name>, based on the default remote branch
- You can also ask Claude mid-session: "work in a worktree"
- Add .claude/worktrees/ to .gitignore to keep git status clean

Cleanup is automatic:

- No changes: worktree and branch are removed on exit
- Changes exist: Claude prompts you to keep or remove
- Manual cleanup: git worktree list / git worktree remove <path>

Subagents can also run in isolated worktrees:

```
# In .claude/agents/migrator.md frontmatter:
---
name: migrator
description: Migrate files to new API
isolation: worktree
---
```

- Each subagent gets its own worktree, cleaned up automatically
- Great for batch migrations and parallel refactors

TIP

For non-git VCS (SVN, Perforce, Mercurial), configure WorktreeCreate and WorktreeRemove hooks to replace the default git behavior.

Ways to Run in Parallel

- claude -w <name> -- each session in its own worktree, run in separate terminals
- /batch <instruction> -- decomposes large-scale changes into 5-30 units, spawns one agent per unit in its own worktree, each opens a PR
- /loop <interval> <prompt> -- schedule recurring tasks inside a session (poll CI, monitor deployments, check builds); session-scoped
- Claude Code desktop app -- visual session management
- claude --remote -- cloud-hosted tasks on Anthropic's servers; /tasks to monitor, /teleport to continue locally
- Agent Teams (experimental) -- coordinated multi-instance sessions with shared task list

Remote Control: Continue from Any Device

Start a session locally and continue it from your phone or any browser. Your full local environment (filesystem, MCP, tools) stays available. Runs on Pro and Max plans only.

```
# Start a Remote Control session
claude remote-control

# Or from inside an existing session:
/remote-control    (shortcut: /rc)
```

- Connect via claude.ai/code or the Claude iOS/Android app
- Press spacebar to show a QR code for quick phone access
- Use /rename first to give the session a descriptive name

TIP

Remote Control keeps Claude running on your machine -- nothing moves to the cloud. Use claude.ai/code (without Remote Control) when you want a fully cloud-hosted session with no local process to keep open.

Agent Teams (Experimental)

Coordinate multiple Claude Code instances as a team. One session acts as team lead; teammates each have their own context window and can message each other directly -- unlike subagents which only report back to the main agent. Enable via the CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS env var.

```
# Enable in settings.json or shell env
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1

# Ask Claude to create a team
Create an agent team: one focused on security,
one on performance, one on test coverage.

# Control teammate display mode
claude --teammate-mode in-process   # all in one terminal
claude --teammate-mode tmux         # split panes (requires tmux)
```

- Teammates coordinate via shared task list and direct messaging
- Shift+Down cycles through teammates in in-process mode
- Require plan approval before a teammate implements: 'spawn X, require plan approval'
- Best for: parallel review, competing hypotheses, cross-layer changes
- Not for: sequential tasks, same-file edits, routine work (token cost scales per teammate)

TIP

Start with 3-5 teammates and 5-6 tasks each. Use subagents for focused tasks; use agent teams when teammates need to share findings and coordinate on their own.

The Writer/Reviewer Pattern

- Session A (Writer): Implement the feature
- Session B (Reviewer): Review in a fresh context
- Session A: Apply the feedback

Subagents

Run in separate context windows, report back summaries.

```
Use subagents to investigate how our auth system
handles token refresh, and whether we have
existing OAuth utilities to reuse.
```

- Research: "use subagents to investigate X"
- Verification: "use a subagent to review this code"
- Background: Ctrl+B to send to background

Headless Mode & Fan-Out

```
claude -p "prompt"                              # One-off
claude -p "prompt" --output-format json         # Structured
claude -p "prompt" --max-budget-usd 2.00        # Cap spend per run
claude -p "prompt" --max-turns 5                # Limit agentic turns
claude -p "prompt" --fallback-model sonnet      # Fallback if overloaded
```

Skills, Hooks & MCP

Skills (Custom Slash Commands)

> *"I use slash commands for every 'inner loop' workflow. Commands are checked into git. Claude and I use /commit-push-pr dozens of times daily."*
>
> -- Boris Cherny, Claude Code creator

Store in .claude/skills/<name>/SKILL.md. Invoke with /<name>.

```
# .claude/skills/fix-issue/SKILL.md
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix: $ARGUMENTS.
1. gh issue view for details
2. Search codebase, implement fix
3. Write tests, commit, push, create PR
```

- Personal skills: ~/.claude/skills/
- Team skills: .claude/skills/ (commit to git)

Advanced Skill Features

- context: fork -- run skill in an isolated subagent with its own context
- user-invocable: false -- Claude auto-invokes it, hidden from the / menu
- $ARGUMENTS[N] or $N -- access arguments by position ($0, $1, $2...)
- ${CLAUDE_SESSION_ID} -- inject current session ID into skill content

Dynamic context injection: run shell commands before Claude sees the prompt.

```
# .claude/skills/pr-summary/SKILL.md
---
name: pr-summary
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---
PR diff: !`gh pr diff`
PR comments: !`gh pr view --comments`
Summarize this pull request.
```

TIP

The !`command` syntax runs shell commands at skill-load time. Output replaces the placeholder before Claude sees anything -- Claude only gets the rendered result. To trigger extended thinking in a skill, include the word "ultrathink" anywhere in the content.

Plugins

Plugins bundle skills, hooks, subagents, and MCP servers into a single installable unit from the community or Anthropic. Run /plugin to browse.

- Install a plugin: /plugin install <name>
- Code intelligence plugins add symbol navigation and automatic error detection
- Plugin skills use a plugin-name:skill-name namespace -- no name conflicts

Hooks

> *"I use a PostToolUse hook to auto-format Claude's code output. This prevents CI errors from minor formatting issues."*
>
> -- Boris Cherny, Claude Code creator

Unlike CLAUDE.md (advisory), hooks are deterministic -- always execute.

- /hooks for interactive configuration
- Ask Claude: 'Write a hook that runs eslint after every file edit'
- Configured in .claude/settings.json

MCP (Model Context Protocol)

Connect Claude to external tools. Configure in .mcp.json.

```
claude mcp add <server-name>
```

- Slack: paste bug threads and say 'fix'
- Databases: run analytics queries in Claude
- Monitoring: read Sentry errors, CI logs
- Design: integrate Figma designs

Custom Subagents

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
tools: Read, Grep, Glob, Bash
model: opus
---
Review for injection, auth flaws, secrets.
```

Permissions

> *"Use /permissions to pre-allow safe commands rather than broadly skipping security checks with --dangerously-skip-permissions."*
>
> -- Boris Cherny, Claude Code creator

- Auto Mode (--permission-mode auto) -- background classifier auto-approves safe local actions, blocks risky ones (force push, mass delete, IAM changes); announced March 2026, requires Team/Enterprise

Boris Cherny's Workflow

Boris created Claude Code and leads the team at Anthropic. Distilled from his X/Twitter threads in January-February 2026.

Setup

- Fast Mode (/fast) -- 2.5x faster Opus 4.6 when speed matters; toggle mid-session
- Model: Opus with extended thinking for everything
- 5 terminal tabs + 5-10 sessions on claude.ai/code + mobile
- Status line showing context usage and current branch

Daily Workflow

- Starts most sessions in Plan Mode
- Iterates on plans, then auto-accept for one-shot implementation
- Uses /commit-push-pr dozens of times daily
- Tags @.claude in PR reviews to update CLAUDE.md
- Voice dictation (fn x2 on macOS) -- 3x faster prompts

Philosophy

> *"A wrong fast answer is slower than a right slow answer. I optimize for total iteration cost, not per-token expense."*
>
> -- Boris Cherny, Claude Code creator

> *"My setup might be surprisingly vanilla! Claude Code works great out of the box. There is no one correct way to use Claude Code: we intentionally build it so you can customize it and hack it however you like."*
>
> -- Boris Cherny, Claude Code creator

Common Pitfalls

The Kitchen Sink Session

Problem: Multiple unrelated tasks in one session. Context full of noise.

Fix: /clear between tasks.

Correcting Over and Over

Problem: Failed approaches pile up. Context is polluted.

Fix: After two corrections, /clear and write a better initial prompt.

The Over-Specified CLAUDE.md

Problem: Too long. Claude ignores half; important rules get lost.

Fix: Prune ruthlessly. Use hooks for non-negotiables.

The Trust-Then-Verify Gap

Problem: Plausible-looking code that doesn't handle edge cases.

Fix: Always provide verification. Can't verify it? Don't ship it.

The Infinite Exploration

Problem: 'Investigate' without scoping. Claude reads hundreds of files.

Fix: Scope narrowly or delegate to subagents.

Skipping Plan Mode

Problem: Jumping straight to code that solves the wrong problem.

Fix: Plan first. The time pays for itself.

Ignoring Context Usage

Problem: Window fills up, Claude starts forgetting.

Fix: Set up /statusline. Use /compact and /clear proactively.

Test-Driven Development

> *"Claude performs best when it has a clear, verifiable target; tests provide this explicit target, allowing Claude to make changes, evaluate results, and incrementally improve its code."*
> -- Anthropic Engineering Blog

TDD is Claude Code's killer discipline. A failing test gives Claude a concrete, measurable goal. Without it, Claude writes code that looks right but may contain subtle bugs. With it, Claude enters an autonomous loop: write code, run tests, analyze failures, adjust, repeat -- until green.

The Red-Green-Refactor Cycle

1. RED: Write a failing test (you, not Claude)

You define what correctness looks like. Claude figures out how to achieve it. This is the critical separation of concerns.

```
# You write this test, or describe it precisely:
Write a FAILING test for validateEmail.
It should reject 'user@.com' and accept 'a@b.co'.
Do NOT write implementation yet.
```

2. GREEN: Let Claude make it pass

Claude implements just enough code to pass the test. The test constrains scope and prevents over-engineering.

```
Now write the implementation to make all tests pass.
Do not modify the tests. Run them after each change.
```

3. REFACTOR: Clean up while green

All tests pass. Now simplify, remove duplication, improve naming.

```
All tests pass. Refactor for clarity while keeping
them green.
```

TDD for Bug Fixes

The most natural TDD use case. Reproduce the bug as a test, then let Claude fix it.

```
Bug: login fails after session timeout.
1. Write a test that reproduces this (should FAIL)
2. Fix the implementation to make it pass
3. Run the full suite to confirm no regressions
```

Why TDD Works So Well with Claude

- Tests give Claude an autonomous feedback loop -- less human intervention
- Failing tests constrain scope -- Claude can't drift or over-engineer
- The full suite catches regressions Claude might not anticipate
- You define correctness; Claude figures out the how

> **TIP**
>
> Beware context pollution: when test-writing and implementation happen in the same context, Claude subconsciously designs tests around the implementation it's planning. For strict TDD, use separate sessions or subagents for each phase.

Kent Beck on AI + TDD

> *"Test-driven development is a superpower when working with AI agents -- unit tests help ensure that AI agents don't introduce regressions."*
>
> -- Kent Beck (creator of TDD)

Enforcing TDD in Your Workflow

- Add 'Always follow TDD: write failing tests before implementation' to CLAUDE.md
- Add 'Never modify tests to make them pass -- only modify implementation' to CLAUDE.md
- Use TDD Guard (github.com/nizos/tdd-guard) -- a hook that blocks implementation without failing tests
- Create a /tdd skill that codifies the red-green-refactor cycle

---

Sources: Official docs (code.claude.com/docs) | Boris Cherny @bcherny on X
Setup thread: x.com/bcherny/status/2007179832300581177
Team tips: x.com/bcherny/status/2017742741636321619
anthropic.com/engineering/claude-code-best-practices
thenewstack.io | nathanfox.net | Kent Beck via Pragmatic Engineer
code.claude.com/docs/en/cli-reference | /en/skills | /en/remote-control | /en/agent-teams